

تمرین عملی هشتم سوال 2

402105727

متین باقری

402105802

فاطمیما تیمارچی

فایل های تغییر داده شده را در پوشه zocker قرار دادم.

(ا)

در ابتدا image رو دریافت کردم :

```
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo docker pull docker.arvancloud.ir/python:3.11
3.11: Pulling from python
281b80c799de: Pull complete
15f14138abe4: Pull complete
378c64c44580: Pull complete
02e37abc533a: Pull complete
caaa3d3a59e2: Pull complete
e8d2e6e0f086: Pull complete
09900b1877af: Pull complete
Digest: sha256:21b0eeea64594fd2ff1fee068328acf1beea6c37320cfacd9b4dd3eb34a2f711
Status: Downloaded newer image for docker.arvancloud.ir/python:3.11
docker.arvancloud.ir/python:3.11
```

(ب)

دستور صورت سوال رو زدم :

```
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo docker inspect docker.arvancloud.ir/python:3.11 | jq -r '.[0].GraphDriver.Data.UpperDir'
/var/lib/docker/overlay2/603c0e9789fce1a49642a7b10a4695feb6daa8f7ebaa6d707bdbc645050f5a0/diff
```

این خروجی همون مسیر نشون دهنده پوشه ای هست که محتوای لایه بالایی image یا همون diff در اون قرار داره .

برای LowerDir هم دقیقاً همین کار رو انجام دادم .

```
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo docker inspect docker.arvancloud.ir/python:3.11 | jq -r '.[0].GraphDriver.Data.LowerDir'
/var/lib/docker/overlay2/825147f32c4c856f423a4b270ab2db8013e52c1d18102b257b3a1185f61b18d3/diff:/var/lib/docker/overlay2/2633cc12f02511a7e032952c8f613c67ef1223ad81436834f8862825f0714a14/diff:/var/lib/docker/overlay2/84794a7414d5bdd105800e3f4a109b6f5680b48a32ff9c800cb8c19577dcd304/diff:/var/lib/docker/overlay2/7410810fd3af71d897ed66d696631107c3b75ed30dba6bc378ff870817f685aa/diff:/var/lib/docker/overlay2/8dd366eb94c2b76f654b0b80be7c8c16583ec145f5ba0c9e03a79e9ae778d506/diff:/var/lib/docker/overlay2/f44266504ec7cb8c376a94e93f33581c09e218ef810b96b3a55434ab16dcf896/diff
```

خروجی مطابق عکس بالا نشون دهنده تمام لایه های image رو نشون میده که به ترتیب از پایین ترین لایه تا نزدیک ترین لایه مرتب شده . این لایه ها تو زمان ساخت فایل سیستم کانترینر به عنوان lowerdir تو OverlayFS استفاده میشن

حالا دستور (sudo ls -alh /var/lib/docker/overlay2/l/) رو زدم تا نحوه نگاشت لایه ها در داکر رو بهم نشون بده و همه symlink ها رو بهم نشون داد که هر کدوم به یک پوشه diff تو مسیر های طولانی اشاره میکرد:

```
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo ls -alh /var/lib/docker/overlay2/l/
total 68K
drwx----- 2 root root 4.0K Jan  5 12:12 .
drwx--x--- 18 root root 4.0K Jan  5 12:12 ..
lrwxrwxrwx 1 root root 72 Oct 22 23:08 3IPDSZYVYN64SALKTCPPFD7YIL3N -> ../179af42a158fa4478beb662ced1be56a04deea2f0f13a55f059a653165117bc0/diff
lrwxrwxrwx 1 root root 77 Dec 14 16:04 6N37UXZ5AT7PX242IYQFT2V34M -> ../16092e19c66d78114a32201d01c8fa79318ac94a3a877bb3f3ac412dd43807e-init/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:08 CIGB2PP0S3BITJW37V6KS064UJ0 -> ../741081fd8a71d897ed66d6966311d7c3b75ed38dba6bc378ff878817f685aa/diff
lrwxrwxrwx 1 root root 72 Dec 14 16:04 G7Z2ZFRLNWT2P3CAHOEH4FM47 -> ../16092e19c66d78114a32201d01c8fa79318ac94a3a877bb3f3ac412dd43807e/diff
lrwxrwxrwx 1 root root 77 Dec 14 16:03 J03ZUHPJ0CTH2133USFHISMUNZ -> ../b66089cffe7ea320420b808e069b8407e6e1110c31c86d9d8eefa7c0584bbe2-init/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:08 KOJ1Q5FUCHULPBVJ2RAG3FYGM4 -> ../8dd366eb94c2b6fe54b0b80be7c8c16583ec145f5ba8c9e03a79e9ae778d506/diff
lrwxrwxrwx 1 root root 77 Oct 27 13:30 MDOAACRDEQ2SGEZXM3CTJYTWJ -> ../feccaa96988b765d86b092d3d7a195d7a6ef1256e11f28749afde4b24d049a1-init/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:12 MUUJ1FNRA2F5YCK6V6ZCDIQCSE -> ../2633cc12f02511a7e032952c8f613c67ef1223ad81436834f8862825f0714a14/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:08 OB5MXSHMS5WCNTCUWYDOWYITW -> ../f44266504ec7cb8c376a94e93f33581c09e218ef810b96b3a55434ab16dcf896/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:12 QWX3Y67EAEYDPL7ACD6N4MH7K7 -> ../603c0e9789f9c1a49642a7b10a4695feb6daa8f7ebaa6d707bdbc645050f5a0/diff
lrwxrwxrwx 1 root root 72 Oct 22 23:00 R33B2YMEWUP5XHD2Y4B55YVY4R -> ../7770dc8b0b7e74b09a6af22df38767c045fdbd19dc0d6f758c59f11c32de1aa/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:12 VBWL0XU30ZKPK6RIJ3PQTFEYNZ -> ../825147f32c4c856f423a4b270ab2db013e52c1d18102b257b3a1185f61b18d3/diff
lrwxrwxrwx 1 root root 72 Jan  5 12:12 XXJ2NZHGWT6TRCON5CWB2RI2J -> ../84794a7414d5bddd105800e3f4a109b6f5680b48a32f9c800cb8c19577dc304/diff
lrwxrwxrwx 1 root root 72 Oct 27 13:30 YOXDZNAMLQSRVFKU5VCVK53U6 -> ../feccaa96988b765d86b092d3d7a195d7a6ef1256e11f28749afde4b24d049a1/diff
lrwxrwxrwx 1 root root 72 Dec 14 16:03 ZEL6EOV24P3E52T4CZ1AT00RDZ -> ../b66089cffe7ea320420b808e069b8407e6e1110c31c86d9d8eefa7c0584bbe2/diff
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo ls -alh /var/lib/docker/overlay2/l/QWX3Y67EAEYDPL7ACD6N4MH7K7
```

حالا یکی رو به عنوان مثال برداشتم و دستور صورت سوال رو اجرا کردم:

```
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo ls -alh /var/lib/docker/overlay2/l/QWX3Y67EAEYDPL7ACD6N4MH7K7
lrwxrwxrwx 1 root root 72 Jan  5 12:12 /var/lib/docker/overlay2/l/QWX3Y67EAEYDPL7ACD6N4MH7K7 -> ../603c0e9789f9c1a49642a7b10a4695feb6daa8f7ebaa6d707bdbc645050f5a0/diff
```

مطابق عکس بالا این symlink نشون میده که داکر برای ساده کردن نام ها از نام های کوتاه شده که به مسیر واقعی diff اشاره میکنه به جای استفاده مستقیم از شناسه های طولانی لایه ها استفاده میکنه.

داکر ما در اصل فایل lower رو به صورت مستقیم تو پوشه مربوط به image ها نداشت به جای اون اطلاعات فایل های زیرین از طریق فیلد LowerDir در خروجی دستور ... docker inspect استفاده شد. این اطلاعات از نظر مفهومی دقیقاً همون محتوای فایل lower هست همچنین در کل داکر image ها رو به صورت لایه ای ذخیره میکنه و هر لایه تو مسیر overlay2/<layer-id>/diff قرار داره و لایه های زیرین از طریق lowerDir و لایه های نهایی از طریق UpperDir قابل شناسایی هستن. همچنین فایل سیستم کانننر در زمان اجرا از طریق OverlayFS ترکیب این لایه ها ساخته میشه.

(ج)

من این دو بخش را با هم انجام دادم و آن ها را با هم تست کردم و همچنین تست تابع فیوناچی هم در همین قسمت انجام دادم.

برای اضافه کردن پرچم base-image- من اومدم فایل های زیر رو تغییر دادم:

1- config.h

در ابتدا برای اضافه کردن پرچم base image اومدم خط زیر رو اضافه کردم در اصل این فیلد میاد اسم image رو نگه میداره و در run.c بری گرفتن lowerDir ما از همین استفاده میکنیم:

```
char base_dir[512];
```

2- main.c

در ابتدا به struct موجود base_image رو اضافه کردم:

```
struct config cfg = {
```

```
.subcommand = NONE,
.name = "",
.command = "",
.base_dir = "",
.base_image = "",
};
```

سپش خطوط زیر رو اضافه کردم . که اگه در دستور --base-image بود اون وقت اسم image رو میخونه و مقدارش رو در cfg.base_image ذخیره میکنه .

```
else if (strcmp(argv[i], "--base-image") == 0) {
    if (i + 1 >= argc) {
        fprintf(stderr,
            "[ERR] Missing --base-image value (e.g. [--base-image "
            "docker.arvancloud.ir/python:3.11]).\n");
        return 1;
    }
    strncpy(cfg.base_image, argv[++i], sizeof(cfg.base_image)
- 1);
    cfg.base_image[sizeof(cfg.base_image) - 1] = '\0';
    i++;
}
```

config.c -3

خطر زیر رو اضافه کردم تا اجرای کانٹینر بدون مشخص کردن base filesystem ممکن نباشه . تا کانٹینر بدون rootfs اجرا نشه :

```
if (cfg.base_image[0] == '\0' && cfg.base_dir[0] == '\0') {
    fprintf(stderr,
        "[ERR] Missing base filesystem. "
        "Use either --base-image or --base-dir\n");
    return 1;
}
```

setup.c -4

در کل باید ساختار setup.c رو overlay می‌کردم برای همین کار های زیر رو انجام دادم :

در ابتدا تابع setup_zocker_dir رو به صورت زیر تغییر دادم تا اینکه اگر tmp/zocker وجود نداشت اون وقت برنامه به جای اینکه خطا بده باید بره خودش ان رو بسازه

```
int setup_zocker_dir(void) {
    if (ensure_dir(ZOCKER_PREFIX, 0755) != 0) return 1;
    return 0;
}
```

سپس تابع setup_container_dir رو به صورت زیر تغییر دادم تا فقط ساختار OverlayFS رو پیاده سازی کنه . حالا برای انکار من نیاز دارم به upperdir برای اینکه تغییرات کانتنینر رو ذخیره کنم . همچنین به workdir برای فضای کاری overlay و همچنین به merged برای root کانتنینر و خروجی نهایی نیاز دارم (در run.c این به عنوان root کانتنینر استفاده میشه) :

```
int setup_container_dir(const char id[64], char
container_dir[256]) {
    const size_t buffer_size = 256;

    if (snprintf(container_dir, buffer_size, "%s/%s",
ZOCKER_PREFIX, id) < 0) {
        return 1;
    }

    // /tmp/zocker/<id>
    if (ensure_dir(container_dir, 0755) != 0) return 1;

    // /tmp/zocker/<id>/upper
    char upper[256];
    if (snprintf(upper, sizeof(upper), "%s/upper", container_dir)
< 0) return 1;
    if (ensure_dir(upper, 0755) != 0) return 1;

    // /tmp/zocker/<id>/work
    char work[256];
    if (snprintf(work, sizeof(work), "%s/work", container_dir) <
0) return 1;
    if (ensure_dir(work, 0755) != 0) return 1;
}
```

```

// /tmp/zocker/<id>/merged
char merged[256];
if (snprintf(merged, sizeof(merged), "%s/merged",
container_dir) < 0) return 1;
if (ensure_dir(merged, 0755) != 0) return 1;

return 0;
}

```

همان طور که مشاهده میکنید من در کد بالا از تابع `ensure_dir` استفاده کردم که علت استفاده از آن هم این بود که ساخت پوشه ها تمیز و تکرار پذیر باشه . کد این تابع به صورت زیر هست . در این تابع همون طور که مشاهده میکنید اول میام بررسی میکنم که آیا `stat(path,&st)` که همون مسیر هست آیا وجود داره یا نه . اگه وجود داشت که با دستور `S_ISDIR(st.st_mode)` میام بررسی میکنم که فایل هست یا دایرکتوری اگه دایرکتوری بود که همه چی حله اما اگه نبود میام میسازمش .

```

static int ensure_dir(const char *path, mode_t mode) {
    struct stat st;
    if (stat(path, &st) == 0) {
        if (!S_ISDIR(st.st_mode)) {
            fprintf(stderr, "[ERR] Path %s exists but is not a
directory\n", path);
            return 1;
        }
        return 0;
    }
    if (errno != ENOENT) {
        fprintf(stderr, "[ERR] stat(%s) failed: %s\n", path,
strerror(errno));
        return 1;
    }
    if (mkdir(path, mode) != 0) {
        fprintf(stderr, "[ERR] mkdir(%s) failed: %s\n", path,
strerror(errno));
        return 1;
    }
    return 0;
}

```

در کل این فایل باید LowerDir ایمپج رو از داکر بگیر و overlay رو mount کنه و chroot رو روی merged انجام بده .

برای اینکار ابتدا چون باید run_container اسم image رو بدونه تا بتونه ... docker inspect بزنه اومدم base_image رو از config (مقدار base-image در config نخیره شده بود) به container اضافه کردم . بنابراین کد این قسمت را به صورت زیر نوشتم :

```
strncpy(c->base_image, cfg.base_image, sizeof(c->base_image));
c->base_image[sizeof(c->base_image) - 1] = '\\0';
```

سپس تابع get_image_lowedir رو به صورت زیر نوشتم چون OverlayFS برای لایه های پایینی نیاز به lowerdir داره و همچنین داکر این مسیر رو در GraphDriver.Data.LowerDir نگه میداره . در این کد ابتدا دستور docker inspect ساخته میشه و سپس خروجی دستور مثل فایل خونده میشه و مسیر LowerDir داخل out نوشته میشه و در نهایت هم اگه خروجی خالی بود یعنی image وجود نداشت یا docker جواب نداده :

```
static int get_image_lowerdir(const char *image, char *out,
size_t out_sz) {
    char cmd[768];

    if (snprintf(cmd, sizeof(cmd),
        "docker inspect --format='{ {.GraphDriver.Data.LowerDir}}' '%s' 2>/dev/null",
        image) < 0) {
        return 1;
    }

    FILE *fp = popen(cmd, "r");
    if (!fp) {
        fprintf(stderr, "[ERR] popen failed for docker inspect\n");
        return 1;
    }

    if (!fgets(out, (int)out_sz, fp)) {
        pclose(fp);
        fprintf(stderr, "[ERR] Failed to read docker inspect output for %s\n", image);
        return 1;
    }
}
```

```

}

pclose(fp);

rstrip(out);

if (out[0] == '\\0') {
    fprintf(stderr, "[ERR] Empty LowerDir for image %s (is it
pulled?)\\n", image);
    return 1;
}

return 0;
}

static int ensure_proc(void) {
    if (mkdir("/proc", 0555) != 0 && errno != EEXIST) {
        fprintf(stderr, "[ERR] Failed to create /proc: %s\\n",
strerror(errno));
        return 1;
    }
    if (mount("proc", "/proc", "proc", 0, NULL) != 0) {
        fprintf(stderr, "[ERR] Failed to mount /proc: %s\\n",
strerror(errno));
        return 1;
    }
    return 0;
}

```

سپس تابع rstrip رو به صورت زیر اضافه کردم . علت اضافه کردنش هم این بود چون خروجی docker inspect.. معمولا \\n داره و ما میخوایم در mount ازش استفاده کنیم برای همین باید \\n و space هارو حذف کنیم:

```

size_t n = strlen(s);
while (n > 0 &&
(s[n - 1] == '\\n' || s[n - 1] == '\\r' ||
isspace((unsigned char)s[n - 1]) || s[n - 1] == '\\') ||

```

```

    s[n - 1] == '') {
    s[--n] = '\0';
    }
    while (s[0] == '\\' || s[0] == '') {
        memmove(s, s + 1, strlen(s));
    }
}

```

سپس داخل run_container خطوط زیر رو اضافه کردم تا منطق ساخت تبدیل شه به overlay و chroot :

```

char container_dir[256];
if (setup_container_dir(cont.id, container_dir) != 0) {
    fprintf(stderr, "[ERR] Failed to setup container
directory for %s\n", cont.id);
    return 1;
}

```

حالا باید مسیر های upper و work و merged رو بسازم تا mount overlay این مسیر هارو بشناسه و ازشون استفاده کنه . علت استفاده از upper و work و merged هم قبلا گفتم بنابراین خطوط زیر رو به کدم اضافه کردم:

```

char upper[512], work[512], merged[512];

if (snprintf(upper, sizeof(upper), "%s/upper",
container_dir) < 0) return 1;
if (snprintf(work, sizeof(work), "%s/work", container_dir)
< 0) return 1;
if (snprintf(merged, sizeof(merged), "%s/merged",
container_dir) < 0) return 1;

```

حالا چون پایه فایل سیستم همون لایه های داکر هست پس lowerdir رو از image به صورت زیر میگیرم :

```

char lowerdir[4096];
if (get_image_lowerdir(cont.base_image, lowerdir,
sizeof(lowerdir)) != 0) {
    fprintf(stderr, "[ERR] Could not get LowerDir for image:
%s\n", cont.base_image);
    return 1;
}

```

مطابق کد زیر در ابتدا با استفاده از opts یک رشته میسازم که آدرس رو نگه میداره و به OverlayFS میگیره که از کجا بخونه و کجا بنویسه . حالا کرنل میاد یک فایل سیستم میسازه که فایل های base image رو از lowdir

بخونه و تغییرات رو تو upperdir مینویشه و نتیجه نهایی رو هم داخل merged نمایش میده و سپس روی merged ما mount overlay میزنم و اگه mount موفق شد اون وقت merged تبدیل میشه به یک view ترکیبی در نتیجه هر چی که داخل lowerdir هست دیده میشه و هر چی هم تو کانتینر تغییر بدیم در upperdir ذخیره میشه ولی در merged هم دیده میشه :

```
char opts[4096];
if (snprintf(opts, sizeof(opts),
    "lowerdir=%s,upperdir=%s,workdir=%s",
    lowerdir, upper, work) < 0) {
    return 1;
}

if (mount("overlay", merged, "overlay", 0, opts) != 0) {
    fprintf(stderr, "[ERR] Failed to mount overlay at %s:
%s\n",
        merged, strerror(errno));
    return 1;
}
```

در نهایت هم merged رو به عنوان root کانتینر استفاده میکنم بنابراین روی merged من chroot زدم کد این قسمت هم به صورت زیر میشه :

```
if (chroot(merged) != 0) {
    fprintf(stderr, "[ERR] Failed to chroot into %s: %s\n",
        merged, strerror(errno));
    return 1;
}
```

main.c -5

برای جلوگیری از مقادیر نامعتبر base_image رو به استراکت اضافه کردم :

```
struct config cfg = {
    .subcommand = NONE,
    .name = "",
    .command = "",
    .base_dir = "",
    .base_image = "",
};
```

سپس این قسمت رو مطابق زیر اضافه کردم تا پرچم --base-image به پارسر آرگومان ها اضافه بشه . این بخش اول وجود مقادیر مربوط به این پرچم رو چک میکنه و سپس اسم image رو ذخیره میکنه :

```

else if (strcmp(argv[i], "--base-image") == 0) {
    if (i + 1 >= argc) {
        fprintf(stderr,
            "[ERR] Missing --base-image value (e.g. [--base-image "
            "docker.arvancloud.ir/python:3.11])).\n");
        return 1;
    }
    strncpy(cfg.base_image, argv[++i], sizeof(cfg.base_image)
- 1);
    cfg.base_image[sizeof(cfg.base_image) - 1] = '\0';
    i++;
} else {
    strncpy(cfg.command, argv[i], sizeof(cfg.command) - 1);
    cfg.command[sizeof(cfg.command) - 1] = '\0';
    i++;
}

```

د، فیونانچی (

عکس این بخش به صورت زیر هست و در ادامه آن هارا توضیح خواهیم داد:

```

Timarchi-Bagheri@debian:~/Downloads/zocker$ make clean
rm -f zocker src/config.o src/main.o src/run.o src/setup.o src/config.d src/main.d src/run.d src/setup.d
Timarchi-Bagheri@debian:~/Downloads/zocker$ make
gcc -Wall -Wextra -std=c99 -MMD -MP -c src/config.c -o src/config.o
gcc -Wall -Wextra -std=c99 -MMD -MP -c src/main.c -o src/main.o
gcc -Wall -Wextra -std=c99 -MMD -MP -c src/run.c -o src/run.o
gcc -Wall -Wextra -std=c99 -MMD -MP -c src/setup.c -o src/setup.o
gcc -Wall -Wextra -std=c99 -MMD -MP -o zocker src/config.o src/main.o src/run.o src/setup.o
sudo setcap cap_sys_admin,cap_sys_chroot+ep zocker
[sudo] password for fatima-t:
Timarchi-Bagheri@debian:~/Downloads/zocker$ sudo ./zocker run --name test1 --base-image docker.arvancloud.ir/python:3.11 /bin/sh
Running container 'test1' with PID: 1
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ hostname
test1
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ cat > fib.py <<'PY'
def fib(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

if __name__ == "__main__":
    n = 20
    print("n =", n)
    print("fib(n) =", fib(n))
    print("first 15 fibs:", [fib(i) for i in range(15)])
py

python3 fib.py
> > > > > > > > \[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ \[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ n = 20
fib(n) = 6765
first 15 fibs: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]

```

در این قسمت باید برنامه رو تست می‌کردم. برا تست در ابتدا دستور `sudo ./zocker run --name test1` رو اجرا کردم و وارد کاننتینر شدم. در ابتدا `ls` زدیم و دیدیم هیچ فایل `dicker daemon` وجود نداره و همین نشون میده که فایل سیستم از `docker image` واقعی اومده.

برای بخش دوم تست هم باید تابع فیبوناچی می‌زدیم تا اثبات کنم که پایتون داخل کاننتینر و در محیط ایزوله بدون مشکل اجرا میشه. همون طور که در عکس بالا مشاهده میکنید من دو نوع تابع فیبوناچی رو امتحان کردم در اولی میبینیم که جمله 20 ام را 6765 چاپ شده و همچنین 15 جمله اول فیبوناچی هم درست چاپ شده که به این معنی است که کد فیبوناچی من درست کار میکنه و در نتیجه کاننتینر من درست کار میکنه.

سوال اول:

در اصل در تمرین 6 داکر ما براساس یک فایل سیستم به دست اومده از ایمپج داکر اجرا میشد و محتوای ایمپج به صورت یک آرشیو استخراج شده بود. و در اصل یک دایرکتوری پایه برای اجرای کاننتینر استفاده میشد برای همین روش قبلی فایل سیستم کاننتینر در اصل یک نسخه کپی شده `image` بود و اصلا ساختار لایه ای نداشت. در این تمرین ما سعی کردیم داکری بسازیم که به واقعیت نزدیک تر هست.

فایل سیستم داکر این تمرین از یک `docker image` واقعی میاد که این همون کاری هست که `docker` با `overlayFS` انجام میده و برخلاف داکر تمرین 6 هست که دستی ما اونو کپی میکردیم. در این روش در داخل / ساختار ما لینوکسی است و میتونیم برنامه های واقعی مثل پایتون(فیبوناچی) در اون اجرا کنیم. در این روش هر چیزی رو که در کاننتینر تغییر بدیم یا ایجاد کنیم در `upperdir` ذخیره میشه و میتونیم اونو پیگیری کنیم و همچنین لایه های `image` دست نخورده باقی میمونن(لایه های زیرین `image` به صورت `read only` هستند). در این روش جدید چند کاننتینر میتونن یک `base image` مشترک داشته باشن و تنها `upper` آن ها مجزا باشد و همین باعث میشه که داکر ما هم سریع تر بشه و هم حجم استفاده شده کمتر بشه. همچنین چون نیاز به کپی فایل های پایه نداریم و فقط یک `mounrt` از `overlayFS` انجام میشه در نتیجه ایجاد کاننتینر سریع تر است. در این روش هر کاننتینر فقط تغییرات خودش رو داره و در نتیجه اگه بخوایم یک کاننتینر رو حذف کنیم کافیه `upperdir` رو حذف کنیم و در نتیجه به `image` پایه آسیب زده نمیشه. در این روش جدید چون لایه های `image` ها `read only` هست و در نتیجه احتمال خراب کردن `base` کم میشه و در نتیجه فایل سیستم امن تر هست.

(۵)

سوال دوم:

`Docker commit` میاد از وضعیت کنونی فایل سیستم یک کاننتینر یک `image` جدید میسازه یعنی هر فایلی که ما نصب یا ویرایش یا ایجاد کنیم رو میاد به صورت یک لایه جدید ذخیره میکنه و در نتیجه میتونیم بعدا از اون `image` یک کاننتینر جدید بسازیم. حالا از مزایاش میتونیم اشاره کنیم به اینکه به صورت پیش فرض میاد هنگام

commit کانتینر رو متوقف میکنه تا snapshot ناسازگار نشه . آگه در ادامه دستور change – بزنینم میاد چند دستور dockerfile مثل cmd,env,.. رو روی image خروجی اعمال میکنه .(رفقار image رو بدون نیاز به نوشتن dockerfile تغییر میده)

کاربرد ها :

- 1- چون آگه ما یک کانتینر داشته باشیم و داخلش چندین چیز نصب کرده باشیم و همچنین چندین فایل رو به صورت دستی تغییر داده باشیم و سپس کانتینر رو ببندیم اون وقت اون تغییرات همه از بین میرن اما میتونیم با docker commit یک image از اون بسازیم و هر وقت خواستیم میتونیم نسخه قبلی رو داشته باشیم و دیگه لازم نیست تغییرات رو بنویسیم یا بازسازی کنیم در نتیجه دیباگ سریع تره
- 2- با زدن docker commit ما یک image موقت میسازیم که برای تسته و میتونیم چندین کانتینر مختلف از اون بسازیم بدون اینکه image اصلیمون خراب بشه و خب همین باعث میشه بتونیم نسخه های مختلف کد خودمون رو امتحان کنیم درحالی که نمیدونیم کنوم نسخه نهاییه . اینکار باعث میشه سرعت بالاتر بره .
- 3- بدون نیاز به dockerfile میتونیم یک کانتینر رو برای یک شخص دیگه بفرستیم .
- 4- آگه کانتینر کرش کنه به جای اینکه ببایم اون رو بازسازی کنیم که همیشه میام از اون وضعیت snapshot میگیریم و به نوعی یک عکس لحظه ای از فایل سیستم میگیریم

چگونگی الهام از معماری لایه لایه : ما در این تمرین اومدیم یک مدل ساختیم که در lowdir لایه های image رو داریم که read-only هست . در upperdir ما تغییرات کانتینر رو ذخیره میکنیم که writable هست و merged رو داریم که دقیقا overlayFS هست . حالا آگه بخوایم commit رو پیاده سازی کنیم باید گام های زیر رو پیش ببریم . در ابتدا باید کانتینر رو متوقف کنیم سپس از اونجایی که در overlay تمام تغییرات کانتینر داخل upperdir هستند پس commit یعنی ببایم یه پوشه بسازیم و محتویات upperdir رو به اون layer کپی کنیم و باید این لایه رو read-only کنیم . حالا این پوشه میشه یک لایه جدید image (یک لایه جدید و دائمی تو image ثبت میشه). از اونجایی که image باید بدونه چه لایه هایی داره و ترتیب این لایه ها چیه (metadata) میایم یک فایل json میسازیم و این اطلاعات رو در اون ذخیره میکنیم . حالا آگه کاربر آن را اجرا کرد میایم تمام layer هارو از metadata میخونم و lowerdir رو براساس اون metadata ها میسازم

سوال سوم:

شباهت :

- هر دو با لایه ارتباط دارند . image از چند لایه immutable (فقط خواندنی) ساخته میشه و container میاد همون لایه هارو استفاده میکنه
- Image و کانتینر هر دوشون فایل سیستم دارند

- هر دو در overlayFS استفاده میشوند
- هر دو پایه اجرا برنامه هستند
- هر دو نقش مهمی در isolation دارن
- هر دو باعث قابل انتقال بودن محیط اجرا میشوند

تفاوت :

- Image یک قالب یا الگو هست در حالی که کانتینر یک نمونه در حال اجرا از image هست
- Image لایه هاش فقط خواندنی هست اما کانتینر علاوه بر لایه های image یک لایه قابل نوشتن که همون upperdir هست رو دارد که تغییرات کانتینر رو نگه میدارد (اگرچه چند تا کانتینر یک ایمج مشترک داشته باشن اون وقت تغییراتشون روی هم اثر نداره یعنی لایه نوشتن کانتینر کاملاً اختصاصیه)
- Image خودش اجرا نمیشه و تنها برای ساخت کانتینر استفاده میشه در حالی که کانتینر یک سری مفاهیم اجرایی مثل namespace و mount و hotname و... غیره داره. در اصل هدف image ساخت کانتینر هست و هدف کانتینر اجرای برنامه هست
- یک image میتونه بین چند کانتینر مشترک باشه اما کانتینر اختصاصی هست
- Commit پل تبدیل کانتینر به image هست. یعنی تغییرات کانتینر به صورت پیش فرض وارد image نمیشه و فقط با docker commit میتونیم این تغییرات رو به image تبدیل کنیم
- Image ها معمولاً طول عمر بالایی دارن و به عنوان الگو ذخیره میشن اما کانتینر ها موقتی هستن و بعد از پایان اجرا حذف میشوند